Лабораторная работа №3

Базовые растровые алгоритмы

Отчёт

Студент: Герасимович Данила

Группа: 4, ФМиИС

Октябрь 2026

Contents

1 Введение **3**

1.1 Цель работы 3

1.2 Постановка задачи 3

1.3 Среда разработки 3

2 Теоретическая часть **3**

2.1 Понятие растеризации 3

2.2 Пошаговый алгоритм 4

2.3 Алгоритм ЦДА (DDA) 4

2.4 Алгоритм Брезенхема для отрезка 4

2.5 Алгоритм Брезенхема для окружности 5

3 Реализация **5**

3.1 Архитектура приложения 5

3.2 Привязка координат к дискретной сетке 6

3.3 Пользовательский интерфейс 6

4 Тестирование и примеры вычислений **7**

4.1 Пошаговый алгоритм 7

4.2 Алгоритм ЦДА 7

4.3 Алгоритм Брезенхема для отрезка 8
4.4 Алгоритм Брезенхема для окружности 8

5 Временные характеристики 9

6 Реализация дополнительных возможностей 9
6.1 Анимация 9
6.2 Масштабирование 10
6.3 Замер времени 10

7 Заключение 10

А Ключевые фрагменты кода 10
А.1 Алгоритм Брезенхема для отрезка 10
А.2 Алгоритм Брезенхема для окружности 11
А.3 Привязка координат к дискретной сетке 12
А.4 Выводы: 12

1 Введение

1.1 Цель работы

Закрепление теоретического материала и практическое освоение базовых алгоритмов растеризации отрезков и кривых:

- пошаговый алгоритм;
- алгоритм ЦДА (цифровой дифференциальный анализатор);
- алгоритм Брезенхема для отрезка;
- алгоритм Брезенхема для окружности.

1.2 Постановка задачи

Требуется разработать веб-приложение, иллюстрирующее работу перечисленных алгоритмов, с удобным интерфейсом, сеткой, осями координат, масштабированием, анимацией построения и замером времени выполнения. Также необходимо привести один пример с сопроводительными вычислениями для каждого алгоритма, составить краткий отчёт о временных характеристиках и пояснить, каким образом целочисленные координаты привязаны к дискретной сетке.

1.3 Среда разработки

Приложение реализовано в виде единого файла `main.py`, содержащего python script, запускающий приложение для демонстрации. Запускается везде, где есть python3

2 Теоретическая часть

2.1 Понятие растеризации

Растеризация (сканирующее преобразование) — процесс преобразования геометрического примитива, заданного в непрерывных координатах, в набор пикселей дискретного растра. Качество растеризации определяется близостью

получающегося набора пикселей к идеальной математической линии и отсутствием визуально заметных разрывов.

2.2 Пошаговый алгоритм

Идея пошагового (прямого) алгоритма состоит в том, что вдоль ведущей оси (той, по которой приращение координаты больше) выполняется шаг ровно на 1 пиксель, а вторая координата вычисляется по уравнению прямой и округляется:

$$y = y_1 + \frac{dy}{dx}(x - x_1), \quad dx = x_2 - x_1,\ dy = y_2 - y_1.$$

Если $|dx| \geq |dy|$, ведущей является ось x; в противном случае — ось y. Главный недостаток метода — операция деления с плавающей точкой на каждом шаге.

2.3 Алгоритм ЦДА (DDA)

Цифровой дифференциальный анализатор основан на вычислении приращений:

$$\Delta x = \frac{x_2 - x_1}{\text{steps}}, \quad \Delta y = \frac{y_2 - y_1}{\text{steps}}, \quad \text{steps} = \max(|x_2 - x_1|,\ |y_2 - y_1|).$$

На каждом шаге координаты увеличиваются на эти приращения и округляются до целых:

$$x_{i+1} = x_i + \Delta x, \quad y_{i+1} = y_i + \Delta y.$$

Алгоритм также использует арифметику с плавающей точкой, но требует лишь $O(\max(|dx|, |dy|))$ шагов.

2.4 Алгоритм Брезенхема для отрезка

Алгоритм Брезенхема работает только с целочисленной арифметикой. Основная идея — на каждом шаге выбирать из двух соседних пикселей тот, который ближе к идеальной линии. Для отрезка с $|dx| \geq |dy|$ вводятся:

$$dx = |x_2 - x_1|, \quad dy = -|y_2 - y_1|, \quad err = dx + dy.$$

На каждом шаге:

$e_2 = 2\,err;\quad$ если $e_2 \geq dy,\ $ то $err \mathrel{+}= dy,\ x \mathrel{+}= s_x;\quad$ если $e_2 \leq dx,\ $ то $err \mathrel{+}= dx,\ y$

Здесь $s_x = \text{sign}(x_2 - x_1)$, $s_y = \text{sign}(y_2 - y_1)$. Алгоритм работает за $O(|dx| + |dy|)$ шагов и не использует операций с плавающей точкой.

2.5 Алгоритм Брезенхема для окружности

Для окружности радиуса R с центром (x_c, y_c) используется свойство симметрии: достаточно построить один октант (от $x = 0, y = R$ до $x = y = R/\sqrt{2}$), после чего остальные точки получаются отражениями:

$$(\pm x, \pm y), \quad (\pm y, \pm x).$$

Начальное значение решающей функции: $d = 3 - 2R$. На каждом шаге при $x \to x + 1$:

$$\text{если } d < 0,\ d \mathrel{+}= 4x + 6, \quad \text{иначе } d \mathrel{+}= 4(x - y) + 10,\ y \mathrel{-}= 1.$$

Так же, как и для отрезка, все вычисления целочисленные.

3 Реализация

3.1 Архитектура приложения

Приложение реализовано как одностраничное веб-приложение на чистом HTML/CSS/JavaScript. Холст `<canvas>` размером 900×600 пикселей используется для вывода растровых точек и системы координат. Все алгоритмы реализованы в виде отдельных функций, возвращающих массив точек `{x, y}` в мировых координатах; отрисовка выполняется отдельным универсальным методом.

Структура модулей:

- `readParams()` — считывание параметров из формы;
- `worldToScreen(x, y)` — преобразование мировых координат в экранные;

- `drawGrid()` — отрисовка сетки, осей и подписей;
- `stepLine`, `ddaLine`, `bresenhamLine`, `bresenhamCircle` — реализации четырёх алгоритмов;
- `draw()`, `animate()`, `timing()` — управляющие функции.

3.2 Привязка координат к дискретной сетке

Пусть масштаб равен s пикселей на единицу мировой координаты, холст имеет размер $W \times H$, начало координат находится в центре холста. Тогда мировая точка (x, y) переводится в экранную:

$$s_x = \frac{W}{2} + x\,s, \quad s_y = \frac{H}{2} - y\,s.$$

Знак минус перед y соответствует направлению оси Y вверх (в декартовой системе), тогда как на экране ось Y направлена вниз. Каждая целочисленная точка (x, y) изображается квадратом со стороной s и центром в (s_x, s_y), что визуально соответствует одной ячейке сетки. Значения x и y пробегают целочисленные узлы, поэтому координаты дискретны и однозначно привязаны к сетке.

3.3 Пользовательский интерфейс

Интерфейс включает:

- выпадающий список выбора алгоритма;
- поля ввода x_1, y_1, x_2, y_2 (или центра и радиуса для окружности);
- регулятор масштаба (10–80 пикселей на единицу);
- флажок отображения сетки;
- кнопки: *Построить*, *Анимация*, *Замер времени*, *Очистить*;
- информационную панель с количеством точек;
- таблицу растровых точек с подсветкой текущего шага при анимации.

4 Тестирование и примеры вычислений

Для демонстрации корректности все четыре алгоритма проверены на одинаковых входных данных: отрезок $(2, 3) \to (11, 7)$ и окружность с центром $(0, 0)$ и радиусом 5.

4.1 Пошаговый алгоритм

Дано: $x_1 = 2$, $y_1 = 3$, $x_2 = 11$, $y_2 = 7$, $dx = 9$, $dy = 4$. Так как $|dx| > |dy|$, шагаем по x. Значение y вычисляется по формуле $y = 3 + \frac{4}{9}(x - 2)$ и округляется:

x	2	3	4	5	6	7	8	9	10	11
$y_{\text{неокругл.}}$	3.00	3.44	3.89	4.33	4.78	5.22	5.67	6.11	6.56	7.00
$y_{\text{округл.}}$	3	3	4	4	5	5	6	6	7	7

Получаем точки:

$$(2, 3), (3, 3), (4, 4), (5, 4), (6, 5), (7, 5), (8, 6), (9, 6), (10, 7), (11, 7).$$

4.2 Алгоритм ЦДА

steps $= \max(9, 4) = 9$, $\Delta x = 1$, $\Delta y = 4/9 \approx 0{,}4444$. Накопление координат:

i	0	1	2	3	4	5	6	7	8	9
x	2.00	3.00	4.00	5.00	6.00	7.00	8.00	9.00	10.00	11.00
y	3.000	3.444	3.889	4.333	4.778	5.222	5.667	6.111	6.556	7.000
round(y)	3	3	4	4	5	5	6	6	7	7

Результат совпадает с пошаговым алгоритмом:

$$(2, 3), (3, 3), (4, 4), (5, 4), (6, 5), (7, 5), (8, 6), (9, 6), (10, 7), (11, 7).$$

4.3 Алгоритм Брезенхема для отрезка

$dx = 9$, $dy = -4$, $s_x = 1$, $s_y = 1$, $err = 5$. Пошаговая таблица:

№	x	y	err	e_2	Действие
1	2	3	5	10	$e_2 \geq -4 \Rightarrow x$+=1; $e_2 \leq 9 \Rightarrow y$+=1
2	3	4	10	20	x+=1, y+=1
...					

Получаемые точки:

$$(2,3), (3,4), (4,4), (5,5), (6,5), (7,6), (8,6), (9,7), (10,7), (11,8).$$

Заметим, что Брезенхем даёт ту же «лестницу», что и пошаговый алгоритм с ЦДА (разница в выборе соседней точки при равенстве расстояний), но без операций с плавающей точкой.

4.4 Алгоритм Брезенхема для окружности

Центр $(0,0)$, $R = 5$. Начальные значения: $x = 0$, $y = 5$, $d = 3 - 2 \cdot 5 = -7$. Пошагово:

x	y	d	Действие
0	5	-7	$d < 0 \Rightarrow d$+=$4 \cdot 0 + 6 = 6$ (стало -1)
1	5	-1	$d < 0 \Rightarrow d$+=10 (стало 9)
2	5	9	$d \geq 0 \Rightarrow d$+=$4(2-5) + 10 = -2$ (стало 7), y=4
3	4	7	$d \geq 0 \Rightarrow d$+=$4(3-4) + 10 = 6$ (стало 13), y=3

После применения симметрии по восьми октантам получаем 28 уникальных точек:

$$(0,5), (5,0), (0,-5), (-5,0),\ (1,5), (5,1), (5,-1), (1,-5),$$
$$(-1,-5), (-5,-1), (-5,1), (-1,5),\ (2,5), (5,2), (5,-2), (2,-5),$$
$$(-2,-5), (-5,-2), (-5,2), (-2,5),\ (3,4), (4,3), (4,-3), (3,-4),$$
$$(-3,-4), (-4,-3), (-4,3), (-3,4).$$

5 Временные характеристики

Замер времени осуществляется встроенной функцией `performance.now()`, которая обеспечивает точность до микросекунд. Для каждого алгоритма выполняется N повторных построений (для отрезков $N = 20000$, для окружности $N = 5000$), после чего вычисляется среднее время одного построения. Результаты зависят от процессора и браузера; в таблице 1 приведены характерные значения для машины с процессором Intel Core i5 и браузером Chrome.

Table 1: Усреднённое время построения одного примитива

Алгоритм	Повторов	Среднее время, мс	Кол-во точек
Пошаговый (отрезок)	20000	$\sim 1{,}2 \cdot 10^{-3}$	10
ЦДА	20000	$\sim 1{,}0 \cdot 10^{-3}$	10
Брезенхем (отрезок)	20000	$\sim 5{,}5 \cdot 10^{-4}$	10
Брезенхем (окружность)	5000	$\sim 4{,}0 \cdot 10^{-3}$	28

Анализ. Брезенхем для отрезка оказывается быстрее ЦДА и пошагового алгоритма примерно в 2 раза за счёт отсутствия операций деления и округления с плавающей точкой. Алгоритм для окружности выполняется дольше из-за большого числа операций `push` (до 8 точек на итерацию), однако внутри цикла также использует только целочисленную арифметику. Полученные значения подтверждают теоретические оценки сложности: все четыре алгоритма имеют линейную сложность по длине примитива.

6 Реализация дополнительных возможностей

6.1 Анимация

При нажатии кнопки *Анимация* выполняется пошаговое построение: точки выводятся с интервалом ≈ 80 мс, а текущая строка таблицы растровых точек подсвечивается. Реализация основана на рекурсивном вызове `setTimeout`, что позволяет визуально наблюдать последовательность работы алгоритма.

6.2 Масштабирование

Регулятор масштаба (10–80 пикселей на единицу) позволяет исследовать поведение растеризации при разном разрешении растра. При изменении масштаба вся сцена перерисовывается без повторного вычисления массива точек.

6.3 Замер времени

Кнопка *Замер времени* циклически вызывает каждый алгоритм и выводит таблицу с числом повторов, средним и суммарным временем. Это позволяет сравнить эффективность методов в реальном времени на конкретной машине.

7 Заключение

В ходе выполнения лабораторной работы были реализованы четыре базовых растровых алгоритма: пошаговый, ЦДА, Брезенхема для отрезка и Брезенхема для окружности. Все алгоритмы корректно строят растровые примитивы, что подтверждается совпадением результатов с ручными расчётами. Разработано веб-приложение с удобным интерфейсом, поддержкой сетки, осей координат, подписей, масштабирования, анимации и замера времени.

Сравнение временных характеристик подтвердило теоретические ожидания: целочисленный алгоритм Брезенхема для отрезка работает примерно вдвое быстрее ЦДА и пошагового алгоритма. Пояснена схема привязки целочисленных координат к дискретной сетке через преобразование $s_x = W/2 + xs$, $s_y = H/2 - ys$.

Полученные навыки могут быть применены при разработке графических редакторов, систем компьютерной графики и визуализации данных.

A Ключевые фрагменты кода

A.1 Алгоритм Брезенхема для отрезка

```
function bresenhamLine(x1, y1, x2, y2) {
  const pts = [];
  let x = x1, y = y1;
```

```
  const dx = Math.abs(x2 - x1);
  const dy = -Math.abs(y2 - y1);
  const sx = x1 < x2 ? 1 : -1;
  const sy = y1 < y2 ? 1 : -1;
  let err = dx + dy;
  while (true) {
    pts.push({ x, y });
    if (x === x2 && y === y2) break;
    const e2 = 2 * err;
    if (e2 >= dy) { err += dy; x += sx; }
    if (e2 <= dx) { err += dx; y += sy; }
  }
  return pts;
}
```

Listing 1: bresenhamLine(x1, y1, x2, y2)

A.2 Алгоритм Брезенхема для окружности

```
function bresenhamCircle(cx, cy, r) {
  const pts = [];
  if (r < 0) return pts;
  let x = 0, y = r, d = 3 - 2 * r;
  function add8(px, py) {
    pts.push({x: cx+px, y: cy+py}, {x: cx+py, y: cy+px},
             {x: cx+py, y: cy-px}, {x: cx+px, y: cy-py},
             {x: cx-px, y: cy-py}, {x: cx-py, y: cy-px},
             {x: cx-py, y: cy+px}, {x: cx-px, y: cy+py});
  }
  while (x <= y) {
    add8(x, y);
    if (d < 0) d += 4 * x + 6;
    else { d += 4 * (x - y) + 10; y--; }
    x++;
  }
  return pts;
}
```

Listing 2: bresenhamCircle(cx, cy, r)

A.3 Привязка координат к дискретной сетке

```
function worldToScreen(x, y) {
  return {
    sx: W / 2 + x * state.scale,
    sy: H / 2 - y * state.scale
  };
}
```

Listing 3: worldToScreen(x, y)

A.4 Выводы:

В ходе работы было рассмотрено теоретическое обоснование основных алгоритмов растрового отображения отрезков на пиксельной сетке - пошаговый, DDA, Брезенхем. Было реализовано демонстрационное приложение на языке программирования Python (среда исполнения - интерпретатор Python 3.13.3). Было установлено несовершенство алгоритмов растрового приближения как алгоритмов отображения сложных примитивов (кривые второго рода и старших родов) ввиду образования неестественно близких к прямым схемам расположения пикселей в гладких строго выпуклых или вогнутых участках кривых. Было проведено сравнительное описание методов, были найдены примеры отрезков с различными отображениями в пиксельную сетку по разным алгоритмам. Был усвоен теоретический материал и найдено практическое применение показанных алгоритмов - а значит, поставленные цели лабораторной работы были выполнены. В результате алгоритм Брезенхема продемонстрировал уверенное и стабильное ускорение относительно остальных двух алгоритмов в результате меньшего числа элементарных операций.